LEAPPs FIELD GUIDE

Extracting, Processing, and Querying Apple Unified Logs from an iOS Device

Apple Unified Logs can show locks, app launches, connectivity, navigation, and more. iLEAPP now parses them natively on Windows, Linux, and macOS. No Mac and no JSON conversion required. Here is the updated workflow.

Alexis Brignoni
Updated August 1, 2026

Live version: leapps.org/blog-post?post=2026-07-29-apple-unified-logs

Here is a data source I do not think examiners can afford to ignore anymore: Apple Unified Logs.

Want to know when an iPhone locked or unlocked? Which app launched? When Wi-Fi, Bluetooth, Airplane Mode, or the flashlight changed state? Whether navigation started, the volume buttons were pressed, the phone changed orientation, or someone took a screenshot? The Unified Logs may know.

Not always. Not forever. But often enough, and with enough detail, that this evidence belongs in the normal iOS examination workflow.

I first wrote about this workflow in May 2025. At the time, LAVA was still coming, the dedicated iLEAPP artifacts were future work, and the native Apple tooling had fewer options. A lot has changed.

Short version: acquire the complete logs, preserve the originals, and let iLEAPP read the tracev3 data straight out of the extraction. Starting with iLEAPP 2026.3.0 there is no Mac in the chain, no rebuilt `.logarchive`, no `Info.plist` surgery, and no monster JSON export unless you want one. Process, then do the analysis in LAVA.

Long version: keep reading.

Download the printable PDF edition.

> **Updated 2026-07-30.** iLEAPP 2026.3.0 parses Apple Unified Logs natively on Windows, Linux, and macOS. The JSON workflow documented further down still works and is still valuable as Apple's own rendering of the data, but it is now the alternate route, not the toll booth everyone has to drive through. The new section below covers the native path and, in the spirit of full transparency, exactly where it differs from Apple's output. If 2026.3.0 is not on the releases page yet when you read this, the same section shows how to run it from source today.

One warning before we start: Unified Log messages change across devices and operating-system versions. A pattern that works on one iPhone is not automatically universal. Validate the findings that matter and correlate them with the rest of the case.

What Apple Unified Logs are

Apple describes the unified logging system as a central store for telemetry from across the operating system. Instead of writing everything into ordinary text files, Apple keeps the data in memory and on disk in its own format. Console, the `log` command, and the OSLog framework are the native ways in.

On an iOS full-file-system extraction, the two paths we care about are:

```
/private/var/db/diagnostics
/private/var/db/uuidtext
```

The `diagnostics` directory holds the `.tracev3` data. The `uuidtext` directory holds support data needed to resolve parts of it. You need both. A `.logarchive` is simply the bundle that brings those pieces together in a form Apple's tools understand.

Here is the catch: these logs are enormous and they do not live forever. Retention depends on time-to-live behavior, log level, storage pressure, and other conditions. There is no safe promise that the evidence will remain for a fixed number of days. **Acquire it early.**

Acquisition options

Collect from a connected device with macOS

If you have the device and a Mac, this is the easiest route. Connect and trust the iPhone or iPad, open Terminal, and run:

```
sudo log collect --device --output "/path/to/case-device.logarchive"
```

If more than one device is connected, recent versions of `log` may also give you `--device-name` and `--device-udid`. Run `log help collect` on the acquisition Mac and save the output with your notes. Apple changes this command. We will come back to that.

Do not get clever and limit the acquisition to a narrow time range just because the archive is big. Collect everything available first. Hash it. Preserve it. Narrow it later.

If the native command is not your route, there are other good options:

- UFADE, Christian Peter's open-source Apple-device acquisition tool. Its **Collect Unified Logs** option writes a finished `.logarchive`, and iLEAPP reads that bundle natively. No reconstruction, no `Info.plist` work, no Mac. More on that below.
- iOS Unified Logs Acquisition, Lionel Notari's macOS acquisition tool, which records device and case information, log statistics, and archive hashes.
- Commercial forensic tools that explicitly preserve the Unified Log components.

Reconstruct a working `.logarchive` from a full-file-system extraction

What if you already have a full-file-system extraction instead of a collected `.logarchive`? Build one from a working copy:

1. Create a new directory with a `.logarchive` extension.
2. Copy the complete **contents** of `diagnostics` and `uuidtext` into its root. Copy the contents, not the two parent directories themselves.
3. Add a compatible `Info.plist` at the root of the package.
4. Test the derivative archive with `log stats` or Console on the analysis Mac.

This used to be the easy part. Create a tiny `Info.plist`, add `OSArchiveVersion`, and move on.

Then macOS 26.4 changed the rules.

Johann Polewczyk did the work to figure out exactly what changed and documented it in The Info.plist File in an Apple Unified Logs .logarchive Package. The old one-key plist is no longer enough on current macOS.

Select the correct `OSArchiveVersion`

First, the version number has to match the operating-system generation represented by the evidence:

macOS evidence	iOS/iPadOS evidence	`OSArchiveVersion`
10.12–10.12.5	10.0–10.2	2
10.12.6–10.13.6	10.3–11.x	3
10.14–11.x	12.x–14.x	4
12.x–26.x	15.x–26.x	5

One important distinction: this table is for selecting the value manually. Johann's current `logarchive_info.py` does not detect the evidence OS and choose among versions 2 through 5. The script is built for contemporary archives and explicitly writes `OSArchiveVersion` 5. If you are reconstructing an older archive, use the appropriate historical value and validate it with an analysis system that supports that generation.

For iOS or iPadOS 15 through 26, that means version 5:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>OSArchiveVersion</key>
  <integer>5</integer>
</dict>
</plist>
```

That was enough on older analysis systems. Starting with **macOS 26.4**, `log show` also expects archive-specific timing and stream metadata.

Johann's current script can generate metadata for four log streams when matching `.tracev3` files are present: `SpecialMetadata`, `SignpostMetadata`, `PersistMetadata`, and `HighVolumeMetadata`.

Each stream's `OldestTimeRef` contains `ContinuousTime`, `UUID`, and `WallTime` values derived from that archive. The script also builds `LiveMetadata` and `EndTimeRef` from the most recent `.timesync` boot UUID and the latest matching catalog continuous time.

If `version.plist` is present, the script can also copy its `Identifier` into `SourceIdentifier` and place available `ttl01`, `ttl03`, `ttl07`, `ttl14`, and `ttl30` values under `SpecialMetadata` → `TTL`.

None of these are generic values. The tempting shortcut would be to copy a working plist from another archive.

Do not do that. Those UUIDs and time references belong to the source evidence.

The good news is that Johann also released logarchive_info.py. It reads the target archive's `.tracev3`, `.timesync`, and `version.plist` files and builds the right `Info.plist` at the archive root. If you are working on macOS 26.4 or later, use it:

```
python3 logarchive_info.py "/path/to/case-device.logarchive"
```

Record the script version or commit you used and hash the generated plist with the rest of the working package.

Remember what this package is: a derivative you created for analysis. Keep the untouched source directories, document how you built it, hash the result, and confirm Apple's tools can read it. When possible, compare the timestamps and record counts against another validated rendering.

If you want this workflow as a picture, Tim Korver has a visual acquisition-and-preservation flow in his Apple Unified Log repository. His Thesis Friday site and CLI cheatsheet belong in your bookmarks too.

Inspect the archive before converting it

Before turning a large archive into a much larger JSON file, get the statistics:

```
log stats --archive "/path/to/case-device.logarchive" \
  --style human --overview
```

Save that output. Later, when you are staring at a database with millions of rows, you will want to know whether the time span and counts line up with the archive you started with.

Apple has also been quietly improving the `log` command:

- `--end` support in `log stats`, allowing statistics for the same bounded period used during conversion.
- `log repack`, which can create a smaller derivative archive containing records that match a predicate.
- Additional collection filtering and selection options.

These options depend on the version of macOS doing the analysis. Record `sw_vers`, save `log help`, and keep the exact commands you ran. If you repack or filter an archive, keep the complete original. Always.

Process the extraction natively with iLEAPP

This is the part I have wanted to write since the original article. Starting with iLEAPP 2026.3.0, the whole conversion detour is optional.

iLEAPP now bundles macos-UnifiedLogs, Mandiant's open source Rust parser for the tracev3 format, and drives it directly against the log data in your extraction. Credit where credit is due: Mandiant built and maintains that parser, it is Apache-2.0 licensed, and it is the reason none of what follows needs a Mac.

Here is the whole workflow now:

1. Download iLEAPP 2026.3.0 or later. Windows, Linux, or macOS. Any of them.
2. Point it at your full file system extraction. Zip, tar, or directory.
3. In **Available Modules**, check the **Unified Logs** module. It comes unchecked on purpose, and I will explain why in a second.
4. Start processing.

Release not on the releases page yet, or you just do not want to wait? Everything in this section is on iLEAPP's main branch right now, and running from source gets you all of it today:

```
git clone https://github.com/abrignoni/iLEAPP.git
cd iLEAPP
pip install -r requirements.txt
python admin/scripts/fetch_unifiedlog_iterator.py
python ileapp.py -t zip -i /path/to/extraction.zip -o /path/to/output
```

Prefer the GUI? `python ileappGUI.py` works the same way. The fetch script downloads the parser build for your platform and verifies its hash before writing a single byte, so you know exactly what you are running. Same code the release will ship, and it costs you what the LEAPPs have always cost: nothing.

That is it. No `.logarchive` reconstruction, no `OSArchiveVersion` table, no `Info.plist` metadata generation, no 30 GB JSON file, and no second computer. iLEAPP finds the `diagnostics` and `uuidtext` data, assembles what the parser needs, and streams the records straight into the LAVA database without writing an intermediate file at all. It handles the Apple-native path layout, the Cellebrite UFED layout where the data partition lands in `filesystem2` with no `private/var` prefix, and a ready-made `.logarchive` folder if that is what you have.

That last case includes the bundle UFADE produces with **Collect Unified Logs**. Point iLEAPP straight at the `.logarchive` as your input and process it. You do not need to place it inside an extraction or rename anything first. iLEAPP recognizes the bundle by what is inside it, the `Persist`, `Special`, `Signpost`, and `HighVolume` stores sitting next to a `timesync` directory, rather than by the folder name. That matters more than it sounds: selecting the bundle itself as the input root strips the `.logarchive` name off every path underneath it, so name matching alone would find every file, copy them all, and still report no data. Content matching is what makes a bare UFADE archive work.

Why does the module come unchecked? Because this is the single biggest job in iLEAPP and I am not going to spend fifteen minutes of your life on every routine run without asking. Reading Unified Logs is a decision. Check the box and it is your decision.

How fast is it?

I will give you the numbers I measured instead of adjectives.

On a 1.16 GB log store holding about 29 million events, Apple's own `log show` took 5 minutes 55 seconds and wrote 26.7 GB of JSON. The native parser processed the same store in 4 minutes 10 seconds and wrote nothing in the middle. It also returned about 5.4 million more records, because `log show` quietly drops activity and statedump entries from its default rendering.

On real evidence, end to end:

Image	Size	Records imported	Wall time	Peak RAM
iOS 17.1 full file system (zip)	24 GB	30,362,747	16 min	under 1 GB
iOS 16.5 Cellebrite UFED (zip)	18.8 GB	19,419,414	9.5 min	under 1 GB

That last column matters. The import streams in batches now, so the memory cost is flat whether the archive holds one million records or thirty. Your laptop can do this.

What the native path does not give you

Nobody tells you what their tools are missing, so let me tell you what mine is.

- **The rendering is not byte-identical to Apple's.** Comparing the same records side by side, about 78 percent match Apple's output exactly, and nearly all of the rest differ only in formatting: pointer values as `1A2B3C` instead of `0x1a2b3c`, floats at full precision instead of rounded, whitespace. None of the message patterns the dedicated artifacts search for are affected by any of this. But if a specific message is going in a report, validate it, and remember `log show` remains Apple's own rendering of Apple's own data.
- **The Trace ID column comes back empty** on native imports. The parser does not emit it. No current artifact queries it.
- **A full file system extraction only holds what survived on the device.** I have an iOS 18.7 extraction where the `Persist` directory, which is where most of the good stuff lives, is simply empty. Native parsing cannot read what the extraction never captured. `log collect` against the device remains the most complete acquisition, which is why the acquisition section above is still the first thing in this article.

One more thing that belongs on the record rather than in a limitations list: the parser version iLEAPP bundles is pinned and hash-verified at build time, and documented in the repository. When someone asks which parser produced a set of records, you can answer.

The JSON route still works

Everything below is the original workflow, and it is still the right call in two situations: you are on an iLEAPP version before 2026.3.0, or you want Apple's own rendering as your working copy. Use Apple's command to render the archive into JSON:

```
log show --archive "/path/to/case-device.logarchive" \
  --style json --info --debug > "/path/to/logarchive.json"
```

The `--info` and `--debug` flags matter. Leave them out and `log show` normally gives you only the default level. That means missing records before the analysis even starts.

Also, make room. A multi-gigabyte `.logarchive` can become a JSON file tens of gigabytes in size. In my original test, a 1.63 GB archive became 29.19 GB of JSON. That is not a typo.

Hash the JSON when it finishes and leave the source `.logarchive` alone.

iLEAPP accepts names matching `logarchive*.json`, so `logarchive-device01.json` is fine. Keep only the intended export in the input directory. You do not want iLEAPP guessing between two 30 GB files.

If you already know the time of interest, `--start` and `--end` can make conversion much faster. Just be honest about what you created: it is a filtered derivative. Document the time zone and bounds, keep the full archive, and compare the result against `log stats` using the same window.

Process the JSON with iLEAPP

Now hand the giant JSON file to iLEAPP:

1. Download the current iLEAPP release.
2. Select the directory containing the `logarchive*.json` file as the input.
3. Select an output directory.
4. In **Available Modules**, check the **Unified Logs** module. It comes unchecked by default; Unified Logs are opt-in now, on both the native and the JSON path. The dependent artifacts run automatically after the raw import.
5. Start processing.

The raw module streams the file instead of trying to load tens of gigabytes into memory. It writes the fields we care about into `_lava_artifacts.db`:

- Timestamp, normalized to UTC
- Row number
- Process image path
- Process ID
- Subsystem
- Category
- Event message
- Trace ID

That database is the working set. The `.logarchive` and JSON still hold fields iLEAPP does not import, so keep them.

And no, iLEAPP does not try to cram eighteen million rows into an HTML report. Good. That would be useless.

The full table is a LAVA-only artifact. Open the completed project in LAVA to filter, tag, and export the records that matter. If you want to write SQL directly, DB Browser for SQLite works too.

Unified Log artifacts currently supported by iLEAPP

When I published the original article, dedicated Unified Log artifacts were something we planned to build.

The future arrived, and then it kept arriving.

> **Updated 2026-08-02.** This section originally counted 133 predicates across 13 artifacts. Four merged pull requests later, the module registers **35 artifacts driven by 235 unique message predicates**, every one of them documented in published research, validated against real extractions on iOS 16.5, 17.1, and 18.7, or both. Where a pattern is documented but has not been seen in our own images, the artifact says so in its notes. The full catalog outgrew this article, so it now lives in its own companion piece: Apple Unified Log Predicates in iLEAPP: The Reference. That is where every pattern, process, source citation, and validation status lives, artifact by artifact.

The short tour of what the 35 artifacts cover:

Evidence area	Artifacts
Human presence and handling	biometric sensor events (Face ID frames, Touch ID finger events), touchscreen events, pocket state, lock status, unlock sessions and method
Communication and input	call events (including keypad tone digits), keyboard activity, dictation, notification interactions
Application activity	executed apps, app focus and lifecycle (cold starts, force-kills), interface navigation
Connectivity	Wi-Fi status, Bluetooth status, Bluetooth pairing, personal hotspot, SIM and cellular state, Airplane Mode, AirDrop
Device state and power	power events (boot and shutdown markers), time change (including manual clock setting), battery state, USB and power connections
Media, audio, and camera	media playback, audio status, audio routes, camera capture, flashlight
Motion and vehicle	motion state transitions, driving state, navigation, CarPlay session
Emergency	Emergency SOS engine
The wide net	the raw import plus the broad **logarchive artifacts** collection, which also carries screenshots, charging, CarPlay connections, orientation, Siri requests, and more without dedicated reports yet

Read this part carefully: “supported” means iLEAPP knows how to look for the message patterns we have researched. It does not mean every iOS version emits the same message, and an empty artifact does not prove an event never happened.

If the dedicated artifact is empty, go wider. Check **logarchive artifacts**, then search the raw **logarchive** table. That is also where the next artifact is waiting to be found.

The live list is always available in the iLEAPP source module, the LEAPPs artifact browser, and the predicate reference.

Querying the database

The dedicated artifacts get you to the obvious hits fast. The raw table is where the fun begins.

Once you find something interesting, pivot into the full logarchive around that timestamp, process, subsystem, or message. Pull the records before and after it. One log line tells you an event happened. The surrounding lines often tell you why.

If you are using a SQLite viewer, confirm the actual table and column names first. Framework output can change.

A basic time-window query looks like:

```
SELECT
    timestamp,
    row_number,
    process_image_path,
    process_id,
    subsystem,
    category,
    event_message,
    trace_id
FROM logarchive
```

```
WHERE timestamp BETWEEN :start_utc AND :end_utc
ORDER BY timestamp, row_number;
```

To search by message and process context:

```
SELECT *
FROM logarchive
WHERE lower(process_image_path) LIKE '%springboard%'
  AND lower(event_message) LIKE '%volume%'
ORDER BY timestamp, row_number;
```

Things I keep in mind while querying:

- Work in UTC unless the case requires a documented local-time presentation.
- Use the row number as a stable secondary sort key when many events share a timestamp.
- Pull a window before and after the event of interest; surrounding messages often explain the action.
- Correlate Unified Logs with Biome, KnowledgeC, application databases, power logs, location sources, and file-system timestamps.
- Treat private or redacted fields, missing log levels, message loss, clock changes, and TTL expiration as limitations.
- Validate high-value patterns on the relevant iOS version and hardware whenever possible.

Research and tools

No one person owns this research, and the target keeps moving. These are the resources I keep close:

- Apple Logging documentation - Apple's overview of the unified logging system.
- Apple OSLog documentation - programmatic access to historical log data.
- iLEAPP - native tracev3 parsing or conversion of the Apple JSON export into LAVA/SQLite output, plus the dedicated Unified Log artifacts.
- macos-UnifiedLogs - Mandiant's open source Rust parser for the tracev3 format. This is the engine behind iLEAPP's native support. Apache-2.0, actively maintained, good stuff.
- LAVA - the LEAPPs viewer for large and standard artifact outputs.
- Lionel Notari's iOS Unified Logs - extensive artifact research, articles, references, and “Unified Logs of the Week.”
- Lionel Notari's acquisition tool - guided collection with reporting, statistics, and hashes.
- Lionel Notari's parsing-tool article - a macOS parser that builds complete and filtered databases, supports custom rules, and performs conversion quality checks.
- Lionel Notari on major `log` command updates - `log repack`, filtering changes, and bounded `log stats`.
- Thesis Friday by Tim Korver - ongoing Apple Unified Log artifact research across iOS, macOS, watchOS, CarPlay, physical interactions, and more.
- Tim Korver's Apple Unified Log repository - acquisition/preservation process flow and CLI material.
- Tim Korver's Apple Unified Log CLI cheatsheet - collection, display, predicates, statistics, event types, and example queries.
- Johann Polewczyk's .logarchive Info.plist research - OSArchiveVersion mapping, the macOS 26.4 metadata requirements, and the reconstruction methodology.
- Johann Polewczyk's logarchive_info.py - generates the evidence-specific `Info.plist` required by current versions of macOS.

- UFADE - open-source Apple-device acquisition by Christian Peter. Its **Collect Unified Logs** output is read natively by iLEAPP.
- DB Browser for SQLite - a general-purpose SQLite query tool.

Before you call it done

- Acquire early and preserve the complete Unified Log data.
- Hash the original `.logarchive`, the exported JSON, and important derivatives.
- Record the acquisition Mac, analysis Mac, OS versions, tool versions, commands, time zones, and filters.
- Use the correct `OSArchiveVersion`; on macOS 26.4 and later, generate the additional evidence-specific plist metadata.
- On the native path, record the iLEAPP version; it pins the exact parser build that produced the records.
- If you take the JSON route, export with `--info --debug` so the working copy is comprehensive.
- Compare database counts and time bounds against native `log stats` where possible.
- Use iLEAPP's dedicated artifacts for fast triage and the full table for context and novel research.
- Validate important patterns; do not interpret an empty parser result as proof of absence.

Apple Unified Logs are not an edge-case data source anymore. Acquire them. Preserve them. Query them. **Interpret them, as Tim Korver rightly suggested.**

And when you find a pattern nobody has documented yet, write it down and send it back to the community. That is how this list grows.

For questions, updates, and my current contact links, visit abrignoni.github.io.